

Chris Kane-Pardy

Mugisha

ECE 540

4/29/2026

Throughout this assignment, I used ChatGPT both to assist in debugging my HDL, RISC-V assembly, and C code and to help me document my errors throughout the project. This report below also went through ChatGPT for final formatting and revision. I would also like to note that I know this document was intended to be a 3-5 page document centered around Part 3, but I opted to include my Part 1 and 2 design logs as well, as I actually struggled quite a bit with them both in the beginning. To look at my Part 3 design, just skip to the "Part 3:" header.

Part 1:

I began by following the instructions in Lab 05 to build the RVfpga-EL2 system in Vivado. The initial setup process involved creating a new RTL project, importing the full src directory, adding the required constraint files, and targeting the Nexys A7-100T FPGA. While the lab instructions were generally clear, I encountered several issues during synthesis and bitstream generation that required debugging.

The first issue occurred during synthesis, where Vivado reported a syntax error in the el2_pdef.vh file related to the use of SystemVerilog constructs (e.g., packed). This was due to Vivado interpreting the file as a standard Verilog header instead of a SystemVerilog file. To resolve this, I manually updated the Source File Properties to mark el2_pdef.vh as a **SystemVerilog file** and ensured it was also set as a **Global Include**, as required by the lab instructions.

A second issue involved undefined AXI-related macros, which were traced to missing include paths. This was initially complicated by file path formatting problems (spaces in directory names), which interfered with Vivado's ability to correctly resolve include directories. I resolved this by cleaning up the project path and

explicitly adding the required PULP include directories in the Vivado settings.

After resolving these issues, synthesis still failed intermittently with parsing and module resolution errors (e.g., missing `clk_gen_nexys`). At this point, I determined that the Vivado project state had become inconsistent. Rather than continuing to patch the existing project, I restarted from a clean project and carefully re-executed all Lab 05 steps, paying close attention to:

- Proper inclusion of all source subdirectories
- Correct classification of header files (.vh)
- Addition of required include directories
- Proper designation of `rvfpganexys` as the top module
- Inclusion of the Boot ROM initialization file (`boot_main.mem`)

Following this clean rebuild, synthesis, implementation, and bitstream generation all completed successfully.

Part 2:

I updated `rvfpganexys.xdc` to map the five pushbuttons to a new signal array `io_btn[4:0]`. In `rvfpganexys.sv`, this signal was added as an inout port and routed into the SoC. Within `veerwolf_core.v`, I declared a second set of GPIO signals (`i_gpio2`, `o_gpio2`, `en_gpio2`) and instantiated a second `gpio_top` module. Five bidirectional tri-state buffers were implemented to connect the pushbuttons, with the unused upper input bits tied low to avoid floating logic.

To integrate the new peripheral, I extended the Wishbone interconnect to support eight slaves and assigned `gpio2` a base address of `0x80001800`. This required updating both `wb_intercon.v` and `wb_intercon.vh`, ensuring that all `wbs_*` concatenations and related signal bundles consistently included the new GPIO2 interface.

During development, I encountered a [DRC MDRV-1] Multiple Driver Nets error caused by incorrectly declaring the pushbutton pins as inputs rather than bidirectional signals. This was resolved by consistently using `inout` across the constraints, wrapper, and core, matching the existing GPIO architecture. Synthesis, implementation, and bitstream

generation were then completed successfully without critical errors, and the original GPIO1 LED/switch functionality continued to operate correctly.

A later debugging session showed that the HDL modifications were functioning as intended. The actual cause of the apparent GPIO2 failure was that PlatformIO was configured to load the wrong bitstream path. After correcting `board_build.bitstream_file` to point to the proper Lab 6 implementation output, the updated design programmed correctly, and GPIO2 worked as expected on hardware.

I then developed assembly test programs to validate the design. One program directly mirrored reads from GPIO2_IN at `0x80001800` to the LEDs, and another implemented a 16-bit counter using GPIO1 for LED output and GPIO2 pushbuttons for control. In the final working version, BTNL slows the count, and BTNR resets the counter to 1. Hardware testing confirmed that the pushbuttons now affect execution as intended.

Part 3:

The RGB LED peripheral was designed as a modular extension to the existing VeeRwolf-based SoC, following the same Wishbone-based architecture used in earlier labs. The design was implemented hierarchically to promote clarity, reuse, and ease of debugging. At a high level, the system consists of three layers: a single-channel PWM generator, a three-channel RGB controller, and a Wishbone slave interface that exposes the functionality to software. This layered approach allowed each component to be independently verified before full system integration.

At the lowest level, `pwm_channel.sv` implements a parameterized PWM generator using an 11-bit free-running counter. The counter continuously increments and wraps naturally, and the output is asserted whenever the counter value is less than a programmable duty-cycle threshold. This design was selected because it provides a direct and intuitive mapping between the duty-cycle value and perceived LED brightness, while remaining simple and fully synthesizable. The 11-bit width was chosen to provide sufficient

resolution (1024 levels) without significantly increasing hardware complexity. An enable signal was included so that the output can be cleanly disabled in hardware, rather than relying on software to force duty-cycle values to zero. A debug output exposing the internal counter was also added to aid in verification and potential debugging.

Building on this module, `rgb_pwm.sv` instantiates three independent PWM channels to control the red, green, and blue components of a single RGB LED. Each channel receives its own duty-cycle input, allowing independent brightness control of each color. A shared enable signal was used to simplify control logic and ensure consistent behavior across all three channels. This design decision reduces the number of control signals required at the system level while still providing full color control. The module effectively abstracts the concept of an RGB LED into a reusable hardware block.

The `wb_rgb_pwm.sv` module integrates two instances of the RGB controller into a Wishbone-compatible peripheral. Each RGB LED is controlled by a 32-bit memory-mapped register containing an enable bit and three duty-cycle fields. This register format was chosen to align with the project specification while also allowing a single write operation to fully configure an RGB LED. The duty-cycle fields are packed into fixed bit ranges, enabling efficient extraction in hardware and straightforward construction in software. In addition to the control registers, read-only registers were included to expose internal PWM counter values for debugging purposes. The Wishbone interface uses a simple one-cycle acknowledge scheme, consistent with other peripherals in the system, ensuring predictable timing and compatibility with the existing interconnect.

A key design consideration was maintaining compatibility with the existing Lab 6 system. The project handout suggested a base address of `0x80001800` for the RGB peripheral, but this address was already used by the GPIO2 pushbutton interface. To preserve the working architecture, the RGB peripheral was instead assigned a base address of `0x80001C00`, following the established address spacing pattern used by other peripherals. This ensured that no existing functionality was disrupted while maintaining a clean and consistent memory map.

Integration into the Wishbone interconnect required careful modification of both `wb_intercon.vh` and `wb_intercon.v`. A complete set of master-to-slave and slave-to-master signals was added for the RGB peripheral, following the exact pattern used for GPIO and GPIO2. The interconnect was extended from 8 to 9 slaves, and the RGB peripheral was inserted into the slave ordering between GPIO2 and UART. This required updating the address match table and all `wbs_*` concatenation bundles. Because the interconnect relies on consistent ordering across multiple concatenated buses, even a small mismatch can cause incorrect routing or synthesis failures. During development, a copy-paste error in these concatenations caused incorrect signal connections, which were identified and corrected. This debugging step highlighted the importance of maintaining strict consistency across all interconnect signal groupings.

Within `veerwolf_core.v`, the RGB peripheral was instantiated and connected to the interconnect using the newly defined Wishbone signals. The lower bits of the address bus were used for internal register selection, consistent with the design of other peripherals. The RGB output signals were then routed through the top-level wrapper `rvfpganexys.sv`, where they were exposed as FPGA outputs.

Software development followed an incremental bring-up strategy to reduce debugging complexity. The initial program verified basic functionality by writing fixed duty-cycle values to the RGB registers and confirming that the LEDs responded correctly. This step validated the correctness of the Wishbone interface, register mapping, and physical connections before introducing more complex control logic.

The application was then extended to support pushbutton-based control using GPIO2. Because pushbuttons produce level signals rather than pulses, edge detection was implemented to ensure that each press resulted in a single increment. This prevented rapid unintended changes when a button was held down. The pushbuttons were mapped to color channels as follows: BTNR for red, BTNU for green, and BTNL for blue. Duty-cycle values were incremented in fixed steps and wrapped when exceeding the maximum value, ensuring continuous adjustability without overflow.

An important refinement was the introduction of independent state variables for each RGB LED. Initially, a single set of RGB values was used, which caused both LEDs to display identical colors. This was corrected by maintaining separate duty-cycle values for each LED and using `sw[15]` to determine which set of values should be modified by the pushbuttons. This approach allows both LEDs to retain their state independently, which more closely matches the intended system behavior.

The BTND pushbutton was implemented to reset the selected LED's duty-cycle values to a minimal nonzero value. Using a value of 1 instead of 0 ensures that the LED remains faintly visible, avoiding a completely off state that could be confused with a disabled output.

Finally, the Lab 7 7-segment display controller was integrated into the application. The display is controlled through two memory-mapped registers: an enable register and a digit register. Because the display expects one hexadecimal digit per nibble, each duty-cycle value was converted into a decimal percentage (0-99) and split into tens and ones digits. These digits were then packed into the appropriate positions in the digit register to match the required display format: red on digits 7-6, green on digits 4-3, and blue on digits 1-0, with digits 5 and 2 intentionally left blank. The enable register was configured with active-low bits so that only the required digits are displayed.

Overall, this design demonstrates a complete hardware/software co-design process. A custom peripheral was developed from basic building blocks, integrated into an existing SoC without disrupting prior functionality, and controlled through a structured software application. Each stage of development was validated incrementally, allowing issues to be identified and resolved efficiently. The final system meets all project requirements and provides a clear example of how custom hardware acceleration can be added to a processor-based system.